Cloud Compute Challenge

- **Goal of the Exercise**

  The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform. You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

- **Requirements**

  Service Details

  The service will be a simple web server that returns a static page.

  You can use **apache2** or **nginx** to serve a simple HTML file.

  The service should be **containerized** and stored in a **Docker registry**.

  The service should be deployed on a **single VM instance** as a docker container.

  The image should be pulled from the registry specified earlier.

  The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

- **Deliverables**

  You are to provide the following as part of your solution:

  - **Dockerfile** to build the image.
  - **Terraform codebase** to deploy the following:
  - Docker registry
  - VPC and all required networking resources
  - EC2 instance
  - Load Balancer

- **Notes**

  Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.

  To push the image to the Docker registry, **build the image locally** before running the Terraform code.

  Once the image registry is created, use Terraform to **push the local image** to the registry.

  Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is

Cloud Compute Challenge

created.

For SSH access:

Use a locally created key.

Add it to the cloud provider key storage.

Attach it to the instance.

Use **Terraform best practices** for file structure and code organization.

Use variables, locals, modules, etc. when appropriate.

Include all tfvars, scripts, and other helper files used to deploy the application.

You may use any cloud provider of your choosing, although **AWS is preferred**.

Use a **free tier account** or a **trial account** to deploy the resources.

-

```
aws_network_interface_attachment resource.


Apply complete! Resources: 2 added, 1 changed, 2 destroyed.

Outputs:

alb_url = "app-alb-929018693.us-east-1.elb.amazonaws.com"
ec2_private_ip = "10.0.2.93"
PS C:\Users\Viqaas\Desktop\Testing\Testbench>
```

Cloud Compute Challenge

main.tf > resource "aws_iam_instance_profile" "ec2_profile"

```tf
module "vpc" {
  source = "./modules/vpc"
}

module "alb" {
  source = "./modules/alb"

  vpc_id             = module.vpc.vpc_id
  public_subnet_ids = [
  module.vpc.public_subnet_id,
  module.vpc.public_subnet_2_id
]
  target_instance_id = module.ec2.instance_id
}


resource "aws_key_pair" "deployer" {
  key_name   = var.key_name
  public_key = file("~/.ssh/id_rsa.pub")
}

module "ec2" {
  source = "./modules/ec2"

  instance_name         = "web-server"
  instance_type         = var.instance_type
  subnet_id             = module.vpc.private_subnet_id
  vpc_id                = module.vpc.vpc_id
  key_name              = var.key_name
  allowed_ssh_ip        = var.allowed_ssh_ip
  alb_security_group_id = module.alb.alb_security_group_id
  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
  user_data             = file("${path.module}/userdata.sh")
}
```

-

Cloud Compute Challenge

```
module "ecr" {
  source = "./modules/ecr"

  repository_name = "my-app-repo"
}

resource "aws_iam_role" "ec2_role" {
  name = "ec2-ecr-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "ecr_readonly" {
  role       = aws_iam_role.ec2_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_instance_profile" "ec2_profile" {
  name = "ec2-profile"
  role = aws_iam_role.ec2_role.name
}
```

-

output.tf > output "ec2_private_ip"

```
output "alb_url" {
  value = module.alb.alb_dns_name
}

output "ec2_private_ip" {
  value = module.ec2.private_ip
}
```

-

provider.tf > provider "aws"

```
provider "aws" {
  region = var.aws_region
}
```

-

Cloud Compute Challenge

- terraform.tfvars > key_name

```
aws_region     = "us-east-1"
instance_type  = "t2.micro"
allowed_ssh_ip = "103.24.20.197/32"
key_name       = "New-key"
```

- $ userdata.sh

```
#!/bin/bash

dnf install -y docker
dnf install -y awscli

systemctl enable docker
systemctl start docker

sleep 20

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 526018540742.dkr.ecr.us-east-1.amazonaws.com

docker pull 526018540742.dkr.ecr.us-east-1.amazonaws.com/my-app-repo:latest

docker run -d -p 80:80 --name web 526018540742.dkr.ecr.us-east-1.amazonaws.com/my-app-repo:latest
```

- variables.tf > variable "key_name"

```
variable "aws_region" {
  default = "us-east-1"
}

variable "instance_type" {
  default = "t2.micro"
}

variable "allowed_ssh_ip" {}

variable "key_name" {
  default = "New-key"
}
```

- Modules: ALB

Cloud Compute Challenge

modules > alb > alb.tf > resource "aws_lb" "alb"

```
resource "aws_lb" "alb" {
  name               = var.alb_name
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb_sg.id]
  subnets            = var.public_subnet_ids

  tags = {
    Name = var.alb_name
  }
}
```

modules > alb > listener.tf > resource "aws_lb_listener" "http_listener"

```
resource "aws_lb_listener" "http_listener" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}
```

modules > alb > output.tf > output "alb_security_group_id"

```
output "alb_dns_name" {
  value = aws_lb.alb.dns_name
}

output "alb_security_group_id" {
  value = aws_security_group.alb_sg.id
}
```

Cloud Compute Challenge

modules > alb > sg.tf > resource "aws_security_group" "alb_sg"

```
resource "aws_security_group" "alb_sg" {
  name        = "${var.alb_name}-sg"
  description = "Security group for ALB"
  vpc_id      = var.vpc_id

  ingress {
    description = "HTTP access"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Cloud Compute Challenge

modules > alb > target-group.tf > resource "aws_lb_target_group_attachme

```
resource "aws_lb_target_group" "tg" {
  name     = "${var.alb_name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id

  health_check {
    path                = "/"
    protocol            = "HTTP"
    matcher             = "200"
    healthy_threshold   = 2
    unhealthy_threshold = 2
    interval            = 30
    timeout             = 5
  }

  tags = {
    Name = "${var.alb_name}-tg"
  }
}

resource "aws_lb_target_group_attachment" "tg_attach" {
  target_group_arn = aws_lb_target_group.tg.arn
  target_id        = var.target_instance_id
  port             = 80
}
```

modules > alb > variables.tf > variable "target_instance_id"

```
variable "alb_name" {
  default = "app-alb"
}

variable "vpc_id" {}

variable "public_subnet_ids" {
  type = list(string)
}

variable "target_instance_id" {}
```

- Modules: EC2

Cloud Compute Challenge

modules > ec2 > ec2.tf > resource "aws_instance" "ec2_instance" > iam_instance_profile

```
resource "aws_instance" "ec2_instance" {
  ami                    = "ami-0f4b77a23eac2378a"
  instance_type          = var.instance_type
  key_name               = var.key_name
  user_data              = var.user_data
  iam_instance_profile = var.iam_instance_profile

  network_interface {
    network_interface_id = aws_network_interface.ec2_ni.id
    device_index         = 0
  }

  tags = {
    Name = var.instance_name
  }
}
```

modules > ec2 > ni.tf > resource "aws_network_interface" "ec2_ni"

```
resource "aws_network_interface" "ec2_ni" {
  subnet_id       = var.subnet_id
  security_groups = [aws_security_group.ec2_sg.id]
}
```

modules > ec2 > output.tf > output "security_group_id"

```
output "instance_id" {
  value = aws_instance.ec2_instance.id
}

output "private_ip" {
  value = aws_instance.ec2_instance.private_ip
}

output "security_group_id" {
  value = aws_security_group.ec2_sg.id
}
```

Cloud Compute Challenge

modules > ec2 > sg.tf > resource "aws_security_group" "ec2_sg"

```
resource "aws_security_group" "ec2_sg" {
  name        = "${var.instance_name}-sg"
  description = "Security group for EC2"
  vpc_id      = var.vpc_id

  ingress {
    description = "SSH access"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.allowed_ssh_ip]
  }

  ingress {
    description     = "HTTP from ALB"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [var.alb_security_group_id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

modules > ec2 > variables.tf > variable "iam_instance_profile"

```
variable "instance_name" {
  default = "web-server"
}

variable "instance_type" {
  default = "t2.micro"
}

variable "subnet_id" {}

variable "key_name" {}

variable "user_data" {}

variable "associate_public_ip" {
  default = false
}
variable "vpc_id" {}

variable "allowed_ssh_ip" {}

variable "alb_security_group_id" {}

variable "iam_instance_profile" {}
```

Cloud Compute Challenge

- Module: ECR

modules > ecr > ecr.tf > resource "aws_ecr_repository" "repo"

```
resource "aws_ecr_repository" "repo" {
  name = var.repository_name

  image_scanning_configuration {
    scan_on_push = true
  }

  tags = {
    Name = var.repository_name
  }
}
```

modules > ecr > output.tf > output "repository_url"

```
output "repository_url" {
  value = aws_ecr_repository.repo.repository_url
}
```

modules > ecr > variables.tf > variable "repository_name"

```
variable "repository_name" {
  default = "web-app-repo"
}
```

- Module: VPC

modules > vpc > igw.tf > resource "aws_internet_gateway" "igw"

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "main-igw"
  }
}
```

Cloud Compute Challenge

modules > vpc > output.tf > output "public_subnet_2_id"

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "public_subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "private_subnet_id" {
  value = aws_subnet.private_subnet.id
}
output "public_subnet_2_id" {
  value = aws_subnet.public_subnet_2.id
}
```

modules > vpc > rt.tf > resource "aws_route_table_association" "private_assoc"

```
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "public-route-table"
  }
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }

  tags = {
    Name = "private-route-table"
  }
}

resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "private_assoc" {
  subnet_id      = aws_subnet.private_subnet.id
  route_table_id = aws_route_table.private_rt.id
}
```

Cloud Compute Challenge

modules > vpc > variables.tf > variable "availability_zone_2"

```
variable "vpc_cidr" {
  default = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  default = "10.0.1.0/24"
}

variable "private_subnet_cidr" {
  default = "10.0.2.0/24"
}

variable "availability_zone" {
  default = "us-east-1a"
}

variable "availability_zone_2" {
  default = "us-east-1b"
}
```

modules > vpc > vpc.tf > resource "aws_nat_gateway" "nat"

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "main-vpc"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet"
  }
}

resource "aws_subnet" "private_subnet" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidr
  availability_zone = var.availability_zone

  tags = {
    Name = "private-subnet"
  }
}
```

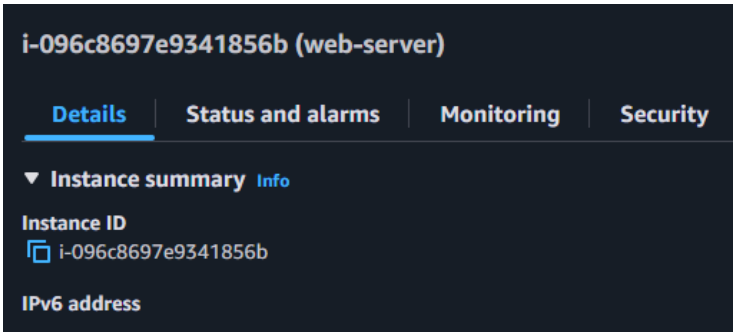
Cloud Compute Challenge

```
resource "aws_subnet" "public_subnet_2" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.3.0/24"
  availability_zone       = var.availability_zone_2
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet-2"
  }
}

resource "aws_eip" "nat_eip" {
  domain = "vpc"

  tags = {
    Name = "nat-eip"
  }
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public_subnet.id

  tags = {
    Name = "main-nat"
  }

  depends_on = [aws_internet_gateway.igw]
}
```

- Dockerfile:

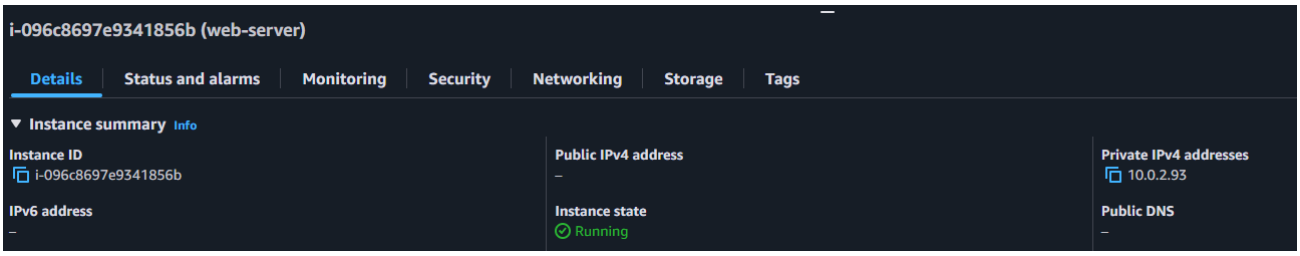
```
Dockerfile > ...
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html
```

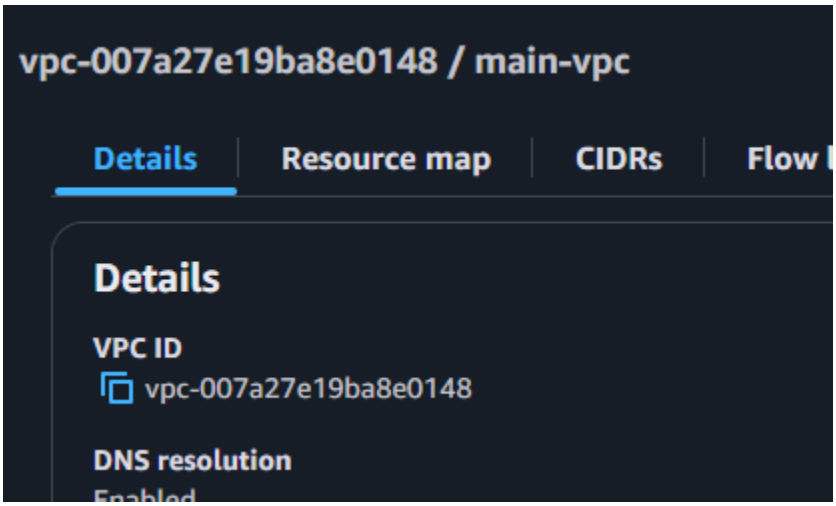
-

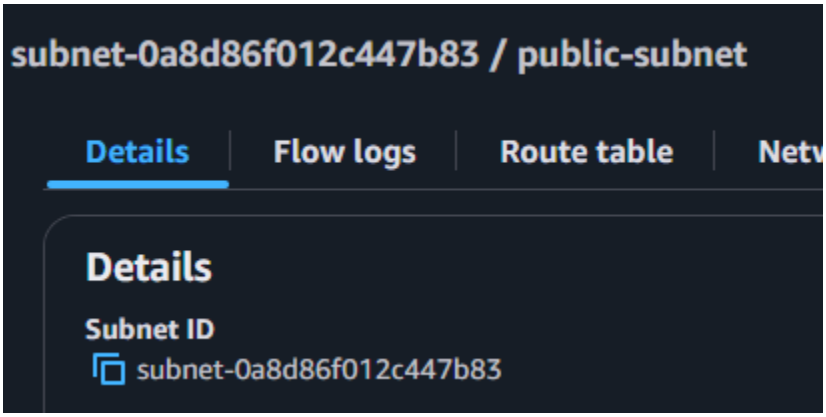


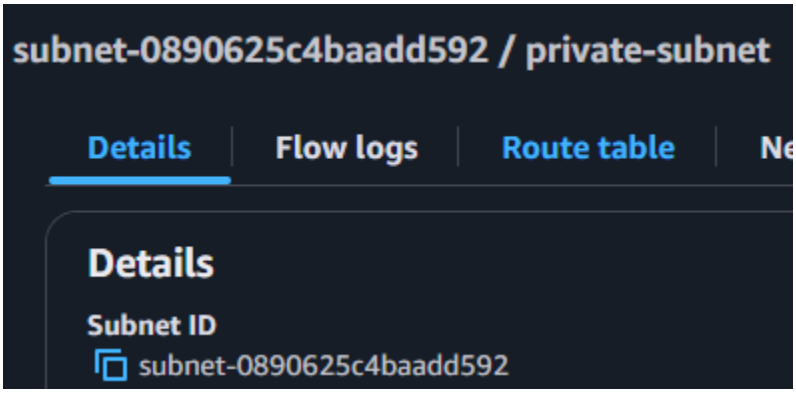
-

Cloud Compute Challenge

- 

- 

- 

- 

-

Cloud Compute Challenge

- rtb-0b53844747201f0bf / private-route-table

Details | Routes | Subnet associations | Edge associations

Routes (2)

Filter routes

Destination	Target
0.0.0.0/0	nat-0d562f517aa343c2e
10.0.0.0/16	local

- rtb-03bd9f59f4fea5e3e / public-route-table

Details | Routes | Subnet associations | Edge associations

Routes (2)

Filter routes

Destination	Target
0.0.0.0/0	igw-06940f0ffdc3912a3
10.0.0.0/16	local

-

Name	Allocated IPv4 addr...
–	44.194.138.66
nat-eip	50.16.183.222
–	54.85.51.244

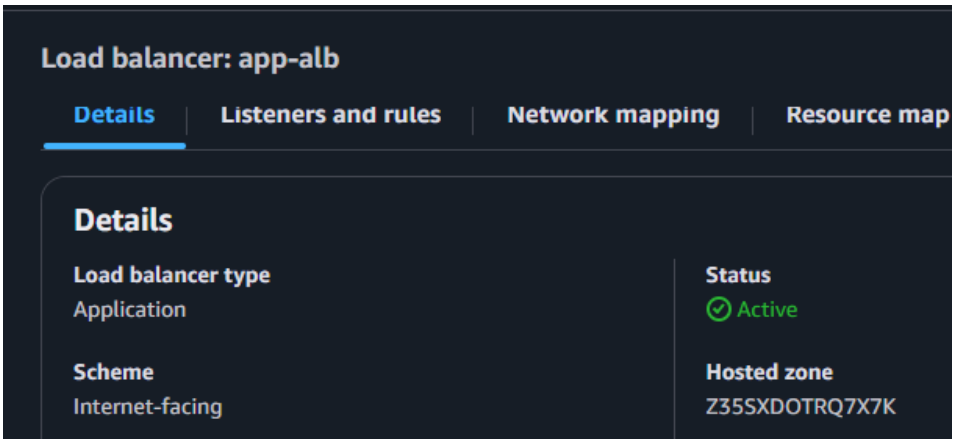
- Network interface: eni-0c17384d31aeb7e58

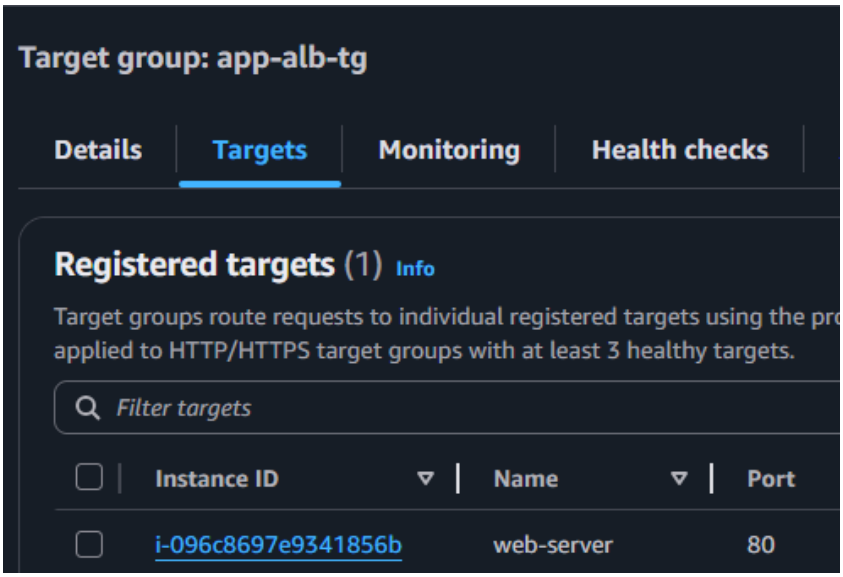
Details | Flow logs | Tags

Network interface details

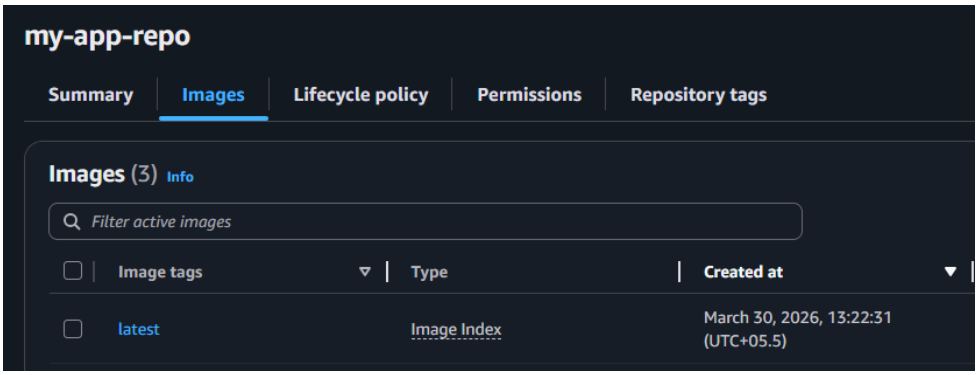
Network interface ID	Name	Description
eni-0c17384d31aeb7e58	-	ELB app/app-alb/57a5d54c94bb8484
Network interface status	**Interface type**	**Security groups**
In-use	Elastic network interface	sg-05d3ec735681804d7 (app-alb-sg)

Cloud Compute Challenge

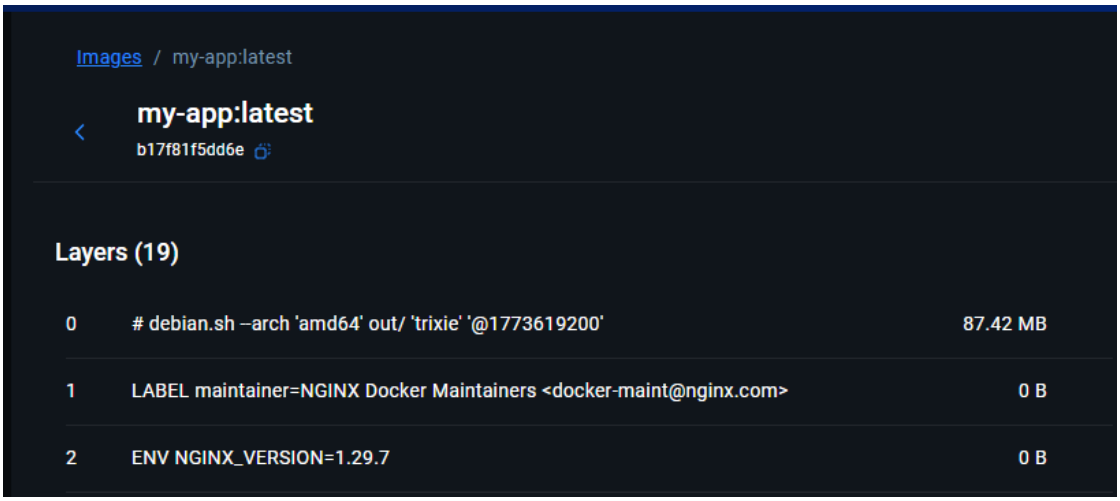
- 

- 

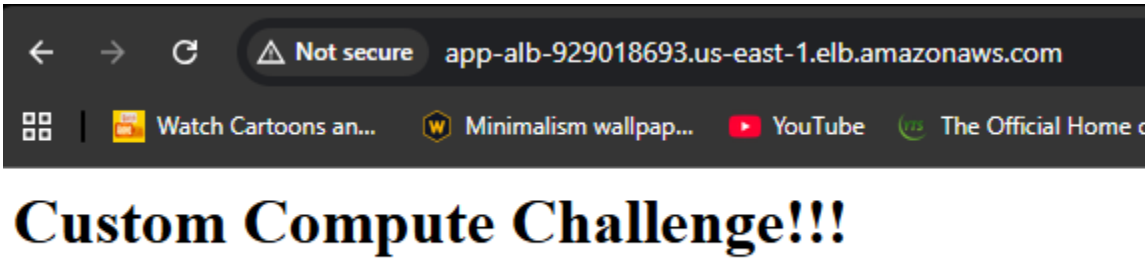
-

- 

Cloud Compute Challenge

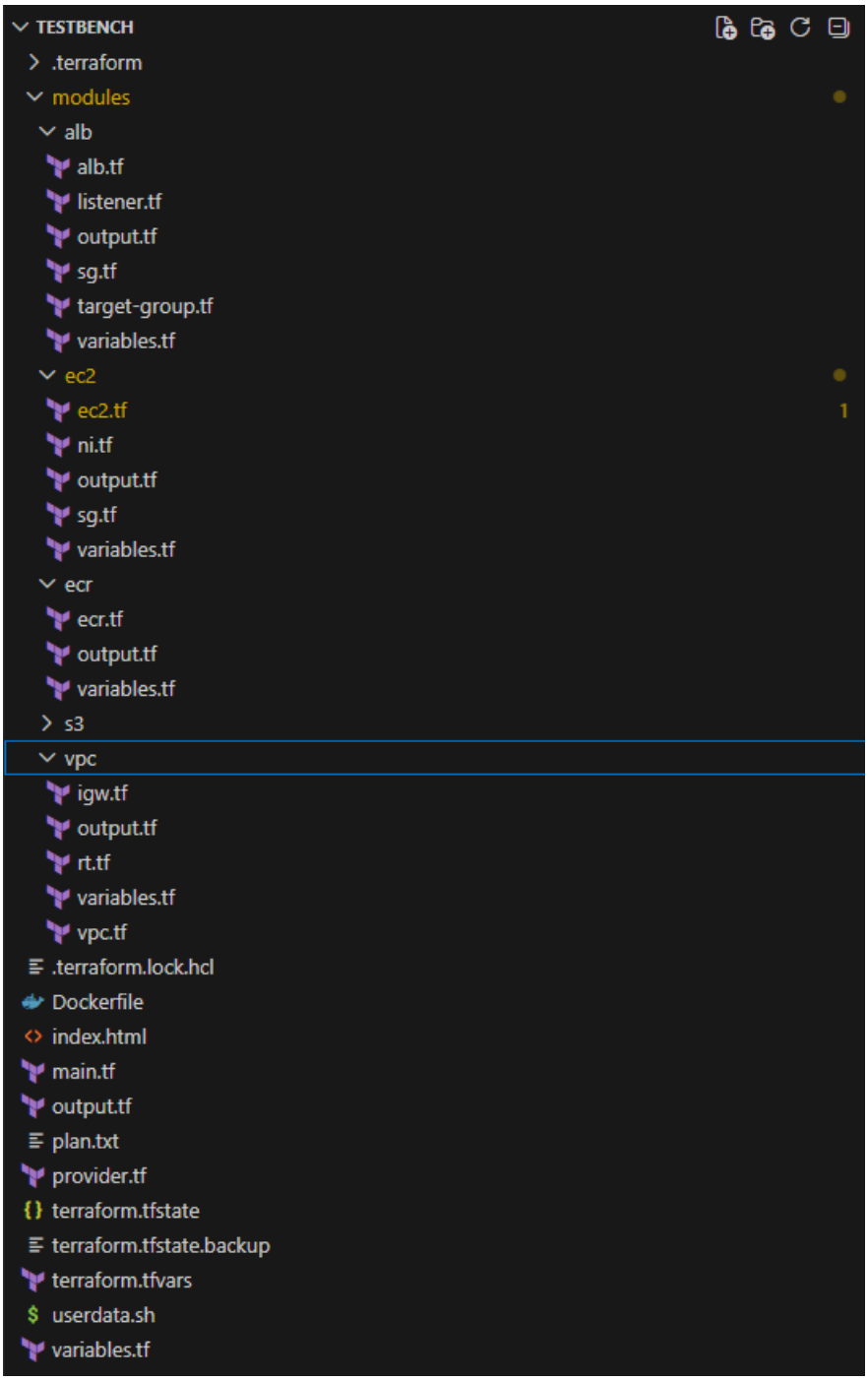


-



Custom Compute Challenge!!!

-

Cloud Compute Challenge

- Terraform Folder Structure: